

What this fraud model is doing

Notes on this repository

Recorded notebooks, seed 42

Abstract

The model learns what a normal transaction looks like, then flags a row it cannot rebuild. This note walks that idea through the two notebooks in the repo, with the numbers those runs actually printed. The synthetic table is generated here. The Kaggle table is the public 2013 card file. Neither result is a production false-positive budget. The derivations sit in Appendix A.

1 The idea

Learn what normal transactions look like. Flag what the model cannot rebuild.

The fraud label never enters the loss. It is used only to keep fraud out of the training and calibration windows, and later to score the holdout. That is normal-only, or semi-supervised, anomaly detection. Calling it unsupervised would be wrong.

2 An autoencoder is a summary

Picture one transaction as a short form someone has to fill in, and then redraw the original from that form alone. The encoder writes the summary. In both notebooks the summary is six numbers. The decoder tries to redraw the amount, the fee, the merchant, and the rest from those six numbers. Training punishes a bad redraw.

Six numbers cannot store every detail of an 18- or 30-wide row, so the network keeps the patterns that show up again and again in the rows it is shown. Those rows are normal. An unusual row asks for a summary the decoder never practiced. The redraw comes out looking more normal than the input, and the gap between them is the score.

This fails in both directions. Fraud that looks ordinary in the columns you kept is easy to redraw and stays under the cutoff. A legitimate row that is merely new can score high. If fraud was mixed into the training normals, the decoder learns to redraw that too, and the gap goes quiet.

A plain autoencoder can also cheat. Nothing tells it where the six numbers should live. It can park a strange row on its own island, far from the summaries of ordinary rows, and still redraw that island perfectly. The training rows look fine. A point halfway between two islands need not decode to anything that looks like a transaction. “How typical is this code?” has no answer, because there is no map of typical codes. The variational piece is that map.

3 What “variational” adds

Instead of pinning each transaction to one exact code, the encoder names a small region: a center, and a width along each of the six directions. A confident row gets a tight region. An unfamiliar row can sit far from the origin, or spread out, or both.

The KL term is a pull that keeps the map tidy. Reconstruction is happy to give a row its own far-away code if that makes the redraw better. The KL charges for a region that wanders away from a simple shared blob, a standard bell curve centered at the origin. Training stops at a compromise. Nearby normal transactions end up in one neighborhood, and a point in that neighborhood is a place the decoder has been asked to turn into a normal-looking row. Figure 1 is the cartoon. It is not a plot from either notebook.

The tidy map is the reason to use a variational autoencoder here. It is not a claim that this model always beats a plain autoencoder at finding fraud. If the bottleneck is already tight, or the anomalies are obvious in the raw columns, the extra pull may change little.

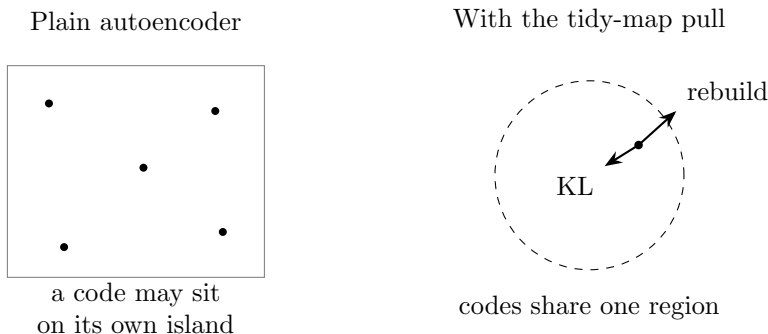


Figure 1: Left: reconstruction alone can isolate a code. Right: the same outward pull, opposed by the KL. The dashed circle is a sketch of the shared blob, not a fitted curve.

4 Randomness the model can still learn from

Training has to try a code drawn from that fuzzy region, and the draw has to teach the center and the width. If the program samples a code and then treats the sample as a fixed number, the learner can improve the decoder but cannot move the region that produced the code. The randomness was a wall.

The fix is to treat the randomness as an input. Draw noise that does not depend on the network, then build the code from the center, the width, and that noise:

$$z = \mu + \sigma \cdot \epsilon.$$

Read it as: start at the center μ , and step by the noise ϵ scaled by the width σ . A hand calculation, not a row from either notebook: center 1, width 0.5, noise -1 , so the code used on that step is 0.5. Because the formula is ordinary arithmetic, a better redraw can nudge the center and the width. The appendix writes the same step with the log-variance the code actually stores.

Scoring does not take that random step. It redraws from the center alone. A new coin flip would change the score of a fixed transaction, and a cutoff fit on one set of flips would not match the next set.

5 One transaction, end to end

The synthetic notebook's checks below were typed by hand. They are not rows the generator sampled. The ordinary one sits near the generator's normal medians. The shifted one breaks the

same links the fraud sampler breaks: a large amount with a small fee, a long gap with high rolling spend, a far-from-home chip purchase, a rare merchant.

Field	Ordinary	Shifted
amount	24.5	280
fee	0.50	1.2
rolling spend, 24h	57	900
distance from home, km	3	80
seconds since last transaction	6600	80000
merchant	m3	m8
device	contactless	chip

Five positive numbers are logged with $\log(1 + u)$ and then z-scored. The mean and spread of that transform are fit on the 5,200 training normals only, and reused everywhere else. Merchant ids `m0–m9` and the devices `chip`, `contactless`, `online` become one-hot blocks in that order. The encoder therefore sees $5 + 10 + 3 = 18$ inputs. Merchant and device are categories. Scoring them with squared error would pretend that `m3` sits halfway between `m2` and `m4`. There is no such order.

The encoder turns those 18 inputs into six centers and six widths. The notebook does not print those twelve numbers for these two checks. What it prints is the redraw error after the decoder has tried to recover the five scaled numbers, the merchant, and the device. Numeric error uses a Huber penalty, gentle on small misses and linear on large ones, so one huge amount cannot dominate. Category error is cross-entropy: did the decoder put its probability on the merchant and device that were actually there?

The score adds those pieces inside the row. Training, by contrast, averages them, so a bigger batch does not inflate the step.

$$\text{score} = \text{sum of the per-feature errors.}$$

The tidy-map pull is not in that sum. Other systems do threshold it. This one does not. The cutoff is a percentile of the same sum on held-out normals, then frozen. A row is flagged when its score is at or above that stored number.

On the ordinary check the score was about 0.052, almost all of it from the numeric columns, and the cutoff was 2.1162, so the row was not flagged. On the shifted check the score was 25.95: about 15.90 from the five numeric terms, 3.79 from the merchant, and 6.26 from the device. The device term is large because a far-from-home chip purchase disagrees with a decoder trained on local chip use. 25.95 is above 2.1162, so the row was flagged.

The Kaggle notebook is the same loop with a different row. There is no merchant and no device, so there is no category term. The thirty inputs are `Time`, the dataset’s anonymized components `V1–V28`, and `Amount`. Amount is logged, then scaled. Time and the components are scaled as they are: Time is a clock starting at 0, and the components are already signed. The split follows time rather than a shuffle: first 60% of rows for training, next 20% for calibration, last 20% for the mixed holdout. The operating cutoff is the 99th percentile of calibration scores, not the 95th.

6 Knobs, including beta

Beta is the price of a code that leaves the shared blob. In the ideal formula that price has weight 1. These notebooks turn the weight down.

Turn it toward zero and the tidy-map pull vanishes. You are close to a plain autoencoder that still happens to emit a center and a width. The summary can improve, and the islands can come back. That is not automatically a bad detector. It does give up the reason for the variational piece.

Turn it up and the blob wins. Every transaction is assigned roughly the same uninformative region, the decoder redraws something close to the average normal row, and the score stops distinguishing anyone. That is posterior collapse: the encoder collapsed onto the prior, and the score went quiet because the code went quiet.

The synthetic run used beta 0.1, a trunk of width 64, and 25 epochs. Epoch 25 printed numeric error 0.2116, category error 0.0861, and a KL of 5.8136. Weighted, the tidy-map term is $0.1 \times 5.8136 = 0.5814$, larger than the two reconstruction pieces together (0.2977), but it had not erased them. Planted fraud was still easy to flag. On that table, 0.1 was a compromise, not a collapse.

The Kaggle columns are a harder case. Once each column is scaled, they are only weakly tied, and six numbers cannot faithfully redraw all thirty. A diagnostic fit on the same training window, seed 42, width 64, and beta 0.1, reached numeric error 0.3456 and a KL of 0.0001 by epoch 4. Predicting the scaled training mean — a vector of zeros — scores 0.3469 on that window. The centers on a training batch had standard deviation about 0.003. The code was unused.

The notebook therefore uses beta 0.01 and width 128. Epoch 12 printed numeric error 0.1382, well under 0.3469, so the code is doing some work. The weighted KL on that epoch is $0.01 \times 7.9357 = 0.0794$, smaller than the numeric term. The curve was still falling slightly at the end (0.1440 at epoch 9, 0.1382 at epoch 12). Twelve epochs is the budget of the run, not a claim that training had finished. Two usual remedies, raising beta gradually and allowing each coordinate a small free KL budget, are not implemented.

The other knobs, in the same spirit:

Knob	Too low	Too high	Recorded choice
beta	islands, prior unused	decoder redraws the mean	0.1 synthetic; 0.01 Kaggle
summary length trunk width	normal rows look odd underfit	room to memorize capacity the code may ignore	6 numbers in both 64 synthetic; 128 Kaggle
Huber kink head weights	almost all linear that head is ignored	huge misses dominate that head swamps the other	1, in scaled units both 1
cutoff percentile epochs, step size	many normal flags error barely moves	misses moderate fraud unstable, or no further gain	95 synthetic; 99 Kaggle 25 / 12 epochs, step 10^{-3}

The percentile is a false-positive budget on normals, not a fraud probability. A 95th percentile aims at flagging about 5% of normal traffic. A 99th aims at about 1%. On a later slice the rate is close, not exact. Synthetic validation normals had their own 95th percentile at 2.040, under the stored cutoff 2.1162, so 4.4% of them were flagged rather than 5%. The Kaggle holdout is later in time. At the 99th percentile its normal flag rate was 0.0063, under the 1% budget.

The table is the Kaggle reconstruction score at four calibration percentiles. Holdout fraud was not used to pick the row. The 99th is the operating point because the 95th flags about 5% of normals (here, 0.0497) on a holdout with 75 frauds among 56,962 rows.

Pct	Threshold	Precision	Recall	F1	Normal FPR
95	14.2839	0.0208	0.8000	0.0405	0.0497
97	18.2676	0.0398	0.8000	0.0758	0.0255
99	34.7925	0.0998	0.5333	0.1681	0.0063
99.5	50.4868	0.1461	0.3467	0.2055	0.0027

From 95 to 99, recall falls from 0.80 to 0.53 and precision rises from 0.02 to 0.10. That trade is what the percentile is.

7 What the runs showed

The synthetic validation report used the calibration cutoff 2.1162. Every one of the 400 fraud rows scored above it. The lowest fraud score was 13.189. Fraud precision is 0.883 because 53 of the 1,200 validation normals also crossed the line, which is the printed flag rate 0.044. Normal median score was 0.667. Fraud median was 24.344.

Class	Precision	Recall	F1	Support
Normal	1.000	0.956	0.977	1200
Fraud	0.883	1.000	0.938	400

Accuracy was 0.967. That number is a sanity check on a planted shift, not a claim about real cards. A pipeline that could not separate scores sitting near 0.7 from scores sitting near 24 would be broken. Separating them does not mean the same weights would catch a subtle fraud.

The Kaggle comparison uses each model’s own 99th percentile on the same calibration window. The forest is 100 trees, each grown on sklearn’s default sample of 256 training rows. Its score is flipped so that larger means more anomalous. Its cutoff was not fit on holdout fraud.

Model	Precision	Recall	F1	PR-AUC	ROC-AUC	Normal FPR
VAE	0.0998	0.5333	0.1681	0.0809	0.9384	0.0063
Isolation Forest	0.0470	0.2267	0.0778	0.0382	0.9528	0.0061

At that shared budget the VAE flags more of the 75 holdout frauds and is more precise: 40 true positives and 361 false positives, against the forest’s 17 and 345. The forest’s ROC-AUC is slightly higher. Both can be true. ROC-AUC asks whether a random fraud outranks a random normal. It can look strong, here about 0.94, while most rows above a useful cutoff are still normal. PR-AUC stays near the base rate, about 0.0013 on this holdout, unless the tail you would actually flag is enriched for fraud. The VAE’s 0.0809 is well above 0.0013, and at the operating point only about one flagged row in ten is fraud.

Accuracy is the wrong headline. The VAE’s accuracy is 0.993. Calling every row normal is $56887/56962 \approx 0.9987$, which is higher, because the model spends 361 false alarms to catch 40 frauds. A silent detector wins on accuracy whenever fraud is this rare.

8 Limits, briefly

The synthetic fraud is far from the normal pattern on purpose. The Kaggle file is a real public extract from one European card program in 2013, and its main columns are already someone else’s

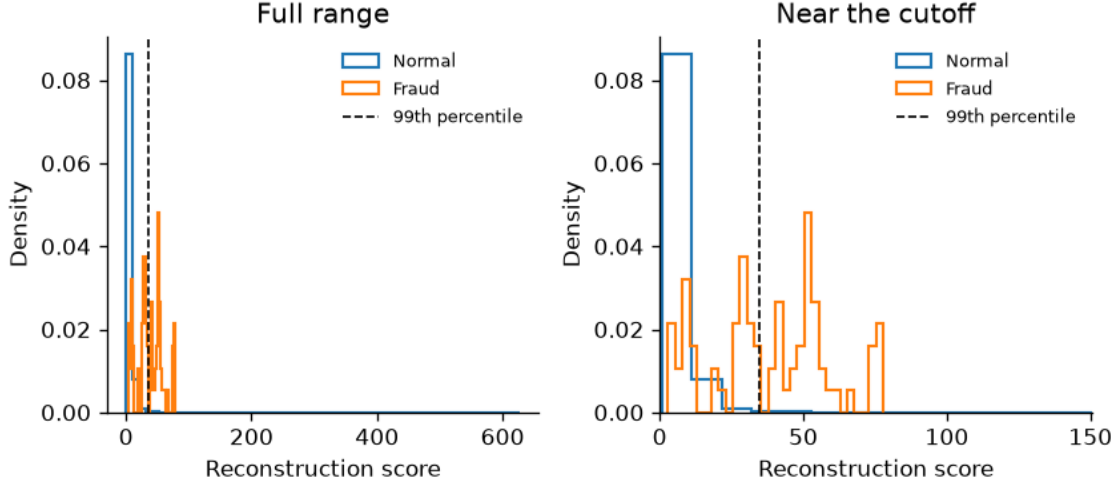


Figure 2: Holdout reconstruction scores from the Kaggle run (seed 42, beta 0.01, width 128, 12 epochs). The dashed line is the calibration 99th percentile, 34.7925.

principal components. Ranking this holdout does not say what a later year or another issuer would do.

Unseen merchant ids are rejected, not scored. There is no cardholder and no sequence; time fields are columns on one row. A wrong “normal” label teaches the decoder to rebuild fraud. The cutoffs 2.1162 and 34.7925 belong to the windows that produced them, and they are not probabilities. Beta was held fixed, and the Kaggle numeric error was still inching down at epoch 12.

A mixed table such as IEEE-CIS would use the synthetic shape — numbers plus ids — at a real vocabulary size, with embeddings once one-hot merchant columns stop fitting. Raising beta gradually is the first training change if a new table collapses the way beta 0.1 did here. A later week needs its own normal calibration window, not a new model class. Serving is the frozen scaler, the weights, and the stored cutoff.

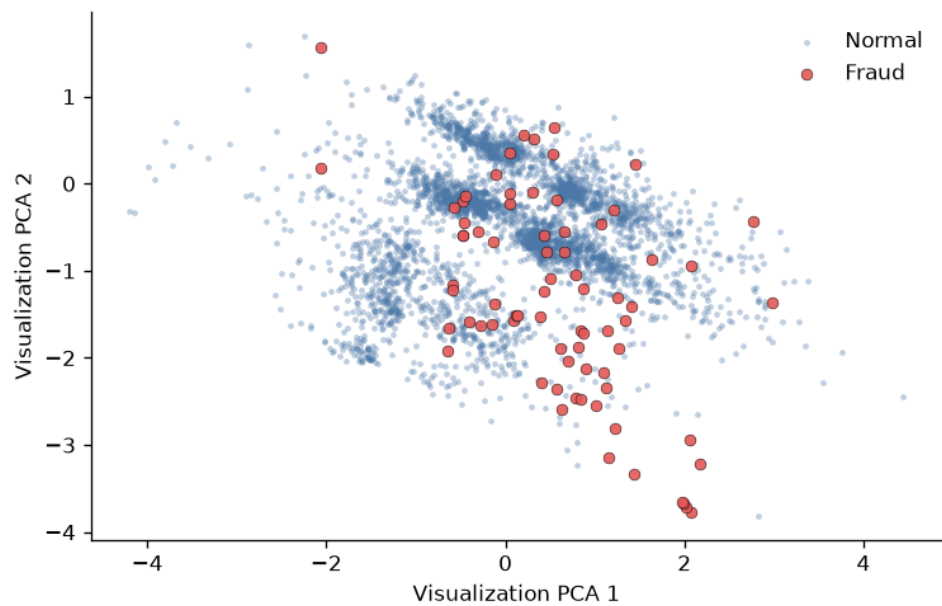


Figure 3: Holdout centers, drawn in two dimensions for display. The axes are a plot PCA fit on 8,000 normal-training centers only (they explained 0.227 and 0.179 of the variance of those centers). The scatter keeps all 75 holdout frauds and 4,000 normals. Many fraud points sit inside the normal cloud. Separation on this plane was not the training target, and these axes are not the dataset's V1 and V2.

A The math

This appendix is the derivation behind the story above. Nothing here changes the recorded numbers.

A.1 Symbols

x is one observed row. z is its code. θ is the decoder's weights. ϕ is the encoder's weights.

Symbol	Meaning
$p(z)$	Prior. Here $\mathcal{N}(0, I)$, fixed before training.
$p_\theta(x z)$	Decoder: how a code would generate a row.
$p_\theta(x)$	Marginal likelihood of a row. An integral over every z .
$p_\theta(z x)$	True posterior. Intractable, because of the marginal.
$q_\phi(z x)$	Encoder: a tractable stand-in for the posterior.
$\mu, \log\text{var}$	Mean and log-variance of the encoder Gaussian.
σ	$\exp(\frac{1}{2}\log\text{var})$, the standard deviation.
ϵ	Noise from $\mathcal{N}(0, I)$, independent of ϕ .
ELBO	Evidence lower bound. The ideal training objective.
β	Weight on the KL term. The ideal ELBO uses $\beta = 1$.

A.2 The integral

A latent-variable model defines the probability of a row by averaging the decoder over every code, weighted by the prior:

$$p_\theta(x) = \int p_\theta(x | z) p(z) dz. \quad (1)$$

If you could compute this, you would train θ by maximizing $\log p_\theta(x)$. You cannot. The code is six-dimensional, and the decoder is linear maps and ReLUs, not a density with a closed integral. A grid over $\mathbb{R}^6$ is not a serious quadrature. Sampling from the prior is the wrong average once training has started: most prior samples are codes the decoder does not associate with this x .

The useful sampler would be the posterior,

$$p_\theta(z | x) = \frac{p_\theta(x | z) p(z)}{p_\theta(x)},$$

and the denominator is (1). The distribution you need in order to train is defined using the quantity you are trying to compute. Variational inference replaces that posterior with a family you choose. Here the family is a diagonal Gaussian. It is not the true posterior.

A.3 The ELBO, two ways

The identity. From $p_\theta(x, z) = p_\theta(x | z) p(z)$ and $\log p_\theta(x) = \log p_\theta(x, z) - \log p_\theta(z | x)$. The left side does not depend on z , so any average over $q_\phi(z | x)$ leaves it unchanged:

$$\begin{aligned} \log p_\theta(x) &= \mathbb{E}_{q_\phi}[\log p_\theta(x, z) - \log q_\phi(z | x)] \\ &\quad + \mathbb{E}_{q_\phi}[\log q_\phi(z | x) - \log p_\theta(z | x)]. \end{aligned} \quad (2)$$

The second expectation is $\text{KL}(q_\phi(z | x) \| p_\theta(z | x))$. It is zero when the two distributions match, and positive otherwise. The first expectation is therefore a lower bound on $\log p_\theta(x)$, and the gap is exactly how far the encoder is from the true posterior.

Expand the joint:

$$\text{ELBO}(x) = \mathbb{E}_{q_\phi}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)). \quad (3)$$

Equation (2) then reads

$$\log p_\theta(x) = \text{ELBO}(x) + \text{KL}(q_\phi(z | x) \| p_\theta(z | x)).$$

The expectation says the code you just drew has to explain the row. The KL against the prior says the encoder cannot use an arbitrary island to do it. When q_ϕ equals the true posterior, the gap is zero and the ELBO equals $\log p_\theta(x)$.

Jensen. Insert q_ϕ , which integrates to one, into (1):

$$\log p_\theta(x) = \log \int q_\phi(z | x) \frac{p_\theta(x | z) p(z)}{q_\phi(z | x)} dz.$$

The log of an average is at least the average of the log, because log is concave:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi}[\log p_\theta(x | z) + \log p(z) - \log q_\phi(z | x)]. \quad (4)$$

The right-hand side is (3). Jensen says the ELBO is a lower bound. The identity names the missing piece, which Jensen hides inside the inequality.

Training never touches $\log p_\theta(x)$. It maximizes a minibatch estimate of the ELBO, or the weighted surrogate written below.

A.4 Closed-form KL for a diagonal Gaussian

The encoder emits μ and logvar in $\mathbb{R}^d$, and

$$q_\phi(z | x) = \mathcal{N}(\mu, \text{diag}(\exp(\text{logvar}))).$$

Coordinate j has variance $\sigma_j^2 = \exp(\text{logvar}_j)$. The prior $\mathcal{N}(0, I)$ has the same independence, so the KL is the sum of one-dimensional KLs.

Drop the index. Up to the shared constant $-\frac{1}{2} \log(2\pi)$,

$$\begin{aligned} \log q(z) &= -\frac{1}{2} \log \sigma^2 - \frac{(z - \mu)^2}{2\sigma^2}, \\ \log p(z) &= -\frac{z^2}{2}. \end{aligned}$$

Under q , $(z - \mu)^2$ has expectation σ^2 , so the second term of $\log q$ averages to $\frac{1}{2}$. And $\mathbb{E}_q[z^2] = \sigma^2 + \mu^2$, so $\mathbb{E}_q[z^2/2] = \frac{1}{2}\sigma^2 + \frac{1}{2}\mu^2$. The KL is the difference of those expectations:

$$\begin{aligned} \text{KL}(q \| p) &= -\frac{1}{2} \log \sigma^2 - \frac{1}{2} + \frac{1}{2}\sigma^2 + \frac{1}{2}\mu^2 \\ &= \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2). \end{aligned}$$

The 2π terms cancelled. With $\sigma^2 = \exp(\text{logvar})$,

$$\text{KL}(q \| p) = -\frac{1}{2} \left(1 + \text{logvar} - \mu^2 - \exp(\text{logvar}) \right). \quad (5)$$

Sum over coordinates, then average over the batch so a short batch and a long batch report the same scale:

$$\text{KL} = \frac{1}{B} \sum_{i=1}^B \left(-\frac{1}{2} \sum_{j=1}^d (1 + \log \text{var}_{i,j} - \mu_{i,j}^2 - \exp(\log \text{var}_{i,j})) \right). \quad (6)$$

Two checks: $\mu = 0$ and $\log \text{var} = 0$ give KL 0, the encoder matching the prior. $\mu = 1$ and $\log \text{var} = 0$ give $\frac{1}{2}$. The code clamps log-variance to $[-10, 10]$ only so the exponential cannot overflow. The clamp is not a model of the data.

A.5 Reparameterization, with the gradient

The reconstruction term averages a function of a sample from q_ϕ . A Gaussian sampler that returns a constant blocks the gradient into μ and $\log \text{var}$. Write the same draw as arithmetic on noise that does not depend on ϕ :

$$\sigma = \exp\left(\frac{1}{2} \log \text{var}\right), \quad \epsilon \sim \mathcal{N}(0, I), \quad z = \mu + \sigma \odot \epsilon. \quad (7)$$

Then $\partial z / \partial \mu = 1$ and $\partial z / \partial \sigma = \epsilon$. On the one-coordinate example in the main text, noise -1 means a push that wants a larger z also shrinks σ on that draw. The next draw can push σ the other way. Across a batch the width learns a scale, rather than chasing one sample. The KL never needs ϵ ; it is already (6). `TabularVAE.forward` is encode, then (7), then decode. Scoring decodes μ .

A.6 Which likelihood becomes which loss

The reconstruction term in (3) is an expected log-likelihood. The loss you minimize is its negative, plus the KL.

A Gaussian decoder with fixed variance has log-likelihood $-\|x - \hat{x}\|^2 / (2\sigma^2)$ plus a constant. Minimizing mean squared error is that likelihood. The variance only rescales it.

A class index matches a categorical likelihood. The decoder emits logits. Cross-entropy is the negative log of the observed class under the softmax. `F.cross_entropy` applies the log-softmax. A softmax in the module as well would train the wrong objective. Squared error on the class index would invent an order the vocabulary does not have.

The notebooks replace Gaussian MSE with Huber loss. With $\delta = 1$,

$$\ell_1(r) = \begin{cases} \frac{1}{2}r^2 & \text{if } |r| \leq 1, \\ |r| - \frac{1}{2} & \text{if } |r| > 1. \end{cases} \quad (8)$$

Inside one scaled standard deviation the penalty is quadratic, the shape of a fixed-variance Gaussian negative log-likelihood. Outside, it is linear, the shape of a Laplace negative log-likelihood. The density whose negative log is (8) is therefore Gaussian in the center and exponential in the tails. No variance parameter is fit. $\delta = 1$ puts the kink at one training standard deviation after scaling.

`VAELoss` does not differentiate the negative of (3) under one joint density. It differentiates

$$\text{total} = \lambda_{\text{cont}} \text{Huber} + \lambda_{\text{cat}} (\text{CE}_m + \text{CE}_d) + \beta \text{KL}, \quad (9)$$

with both lambdas equal to 1. Reductions:

- Huber: mean over the batch and over the continuous columns.

- Each cross-entropy: mean over the batch. The head-means are then added, so each categorical column contributes once. On the Kaggle model there are no heads, the categorical term is a zero tensor, and λ_{cat} multiplies zero.
- KL: (6).

A uniform guess on the synthetic heads would score $\ln 10 + \ln 3 \approx 3.401$. Epoch 1 printed cross-entropy 3.2698, just under chance. Epoch 25 printed 0.0861. The categories were being reconstructed by the end of that run.

Scoring uses the same δ and the same lambdas, summed inside the row, with no KL:

$$\text{score} = \lambda_{\text{cont}} \sum_j \ell_1(\hat{u}_j - \tilde{u}_j) + \lambda_{\text{cat}} \sum_{\text{heads}} \text{CE}(\text{logits}, c). \quad (10)$$

Arithmetic on the definition, not a data row: Huber pieces 0.2 and 0.4. Training averages them to 0.3. The score adds them to 0.6. The synthetic score has five Huber terms and two cross-entropies. The Kaggle score has thirty Huber terms and no cross-entropy. A large KL, the prior density of the center, or a density fit on normal codes would be alternative anomaly signals. They were not the ones implemented in `score.py`.